# From ARC-AGI-3 to NetHack

## Where code strategies meet their limits — lessons, failures, and what came next

researchy / Origin Aleph · agentic research toward NetHack

### Where this approach already won — ARC-AGI-3, source-free, beats prior SOTA

**Result.** The first **perfect** ARC-AGI-3 result — **25/25 games · 183/183 levels**, human-efficient, beating prior state of the art (solved by Claude Fable 5).

**What the approach did (TL;DR).** The agent learned each game's rules *by acting* — no source code — then **synthesized its own** `predict(frame, action)` **world model**, kept only when it exact-matched held-out transitions (verification gate), reasoned the win condition, and replayed the winning moves. Zero training data; deterministic, inspectable code. *The map is the model.*

**How OpenWorld was used.** The **github.com/quome-cloud/openworld** framework runs the loop end-to-end: perceive pixels → explore → **synthesize verified code dynamics** → reason the objective → serve as a live inference server; unsolved games loop back and re-synthesize.

**The point of this report.** This same recipe **beat or matched BALROG SOTA on four of six worlds** (MiniHack 92.5% vs 40%, TextWorld 90% vs 75.7%, Baba & BabyAI 100%). It falls short on the two **stochastic-survival** worlds — Crafter and **NetHack** — and what follows is *why*, and what we discovered to do instead.

## Executive summary

We built verified, executable world models that solved ARC-AGI-3 *source-free* — a perfect 25/25 games, beating prior SOTA (see call-out above; openworld) — and the same recipe **beat or matched BALROG SOTA on four of six worlds** (MiniHack, TextWorld, Baba, BabyAI). We then turned it on NetHack and hit a wall: **23 successive code interventions produced no improvement on the benchmark mean.** This report explains why — the difference is not effort but the *shape of the world*. Code strategies excel when a world is deterministic and fully representable in code; they degrade under **randomness** and collapse under **combinatorial explosion** of the action/item space. NetHack has both. We distill five lessons, document the failures as case studies, and highlight the approaches that finally showed promise: **LLM-in-the-loop reasoning** (a 92% survival-choice rate where fixed rules died), **class specialization via re-rolling**, and **structured belief** (a possibility tensor + Sudoku-style deductive item ID).

## 1 · Two very different worlds

**ARC-AGI-3** puzzles are effectively *single-level, deterministic, and fully code-expressible*: the entire world state and the effect of every action can be written down, so a search or a compiled rule finds the solution. **NetHack** is the opposite on every axis — dozens of dungeon levels (needs **memory**), pervasive **randomness** (a door takes ~10 kicks; searching for a hidden passage takes an unknown number of tries; combat and item identities are randomized *per game*), and a **combinatorial** action space (any action can be applied to any item, and items interact — an *action* × *item* × *item* matrix that cannot be enumerated). The same code-strategy playbook that wins ARC-AGI-3 has no purchase here.

# 2 · The BALROG spectrum — where the same approach *did* work

BALROG has six worlds, and we ran the same OpenWorld recipe on all of them. The result is the cleanest possible evidence that the wall is the *world*, not the method: **we beat or matched the leaderboard SOTA on four of the six. MiniHack 92.5%** vs 40% SOTA (+52.5pp, more than 2×); **TextWorld 90%** clean / 100% privileged vs 75.7% (+14.3pp); **Baba Is AI 100%** (saturated); **BabyAI 100%** (matched SOTA). These four are deterministic, largely code-representable worlds — grid instruction-following, rule-manipulation puzzles, parser text games — exactly the ARC-AGI-3 regime.

The two worlds where we fall short are precisely the **stochastic, survival** ones: **Crafter 47.3%** (below 57.3% SOTA) and **NetHack 6.09** (statistically tied with 6.8% SOTA — the one world we cannot dominate). In both, our own analysis finds the binding constraint is **death, not competence** — the planner saturates the logic but keeps dying to randomness. The thesis, in one line: **NetHack (and Crafter) simply have more probability and luck and a vastly larger action/scenario space, which make the deterministic code-strategy approach intractable.** A telling detail: **MiniHack runs on NetHack's own engine**, scoped to small deterministic tasks — and we score 92.5% there. The engine isn't the problem; the full game's stochastic, combinatorial scope is.

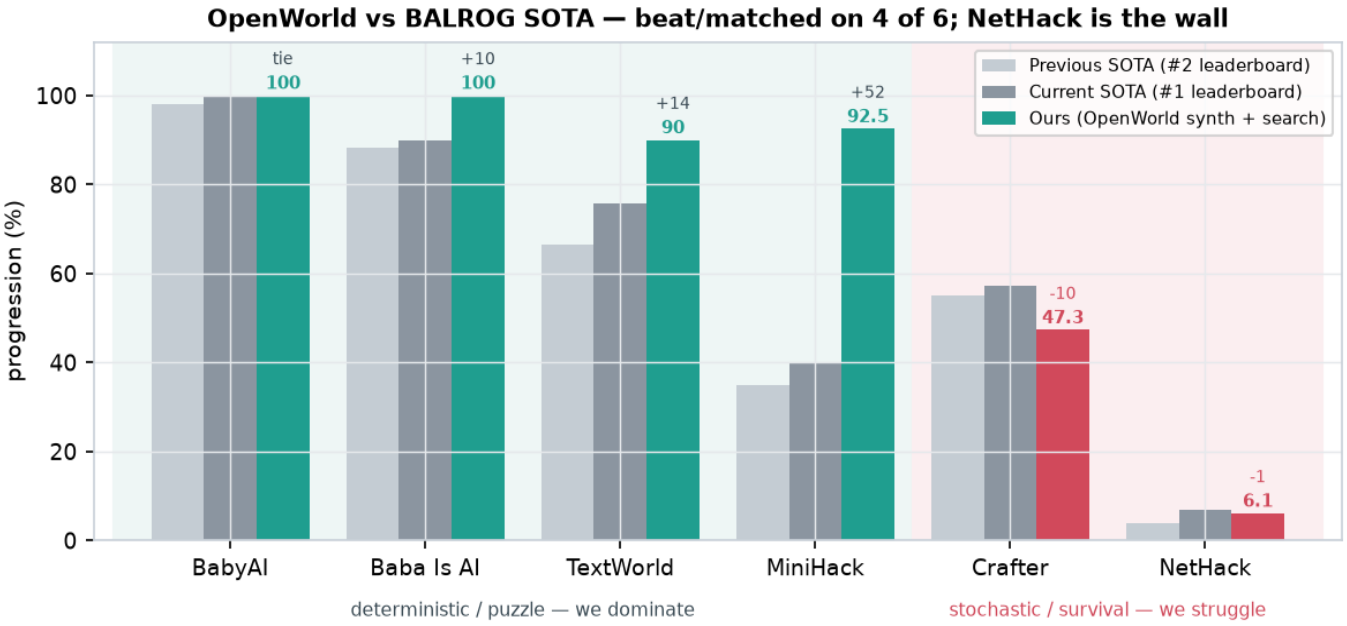


Fig 1 — Our results vs BALROG SOTA across all six worlds. We beat/match SOTA on the four deterministic worlds (teal); we fall short only on the two stochastic-survival worlds (red: Crafter, NetHack), where death is the binding constraint. NetHack bars are small because the whole field scores single digits — 6.09 vs 6.8.

# 3 · Five lessons

**Lesson 1. Code strategies work when the world is deterministic and code-representable.**

This is the ARC-AGI-3 regime and it is real: if you can write the world down, you can write the solution down. The failure mode is assuming it generalizes.

**Lesson 2. Randomness raises the difficulty — and must be modelled, not ignored.**

Stochastic actions (kick ≈ 10 tries, search ≈ N tries) mean a strategy is a distribution over outcomes, not a lookup. Hard rules that assume determinism fire at the wrong time and waste limited resources (our forensic 'pray-early' fix

hurt survivors for exactly this reason).

**Lesson 3. Combinatorial explosion defeats enumeration.**

The action×item×item matrix is too large to code cell-by-cell. Most cells are useless, a few are game-winning, and which is which depends on context. A fixed policy cannot carry the whole matrix; it needs a way to *reason* over the relevant slice.

**Lesson 4. Memory matters.**

A multi-level world requires remembering what/where things are across levels and across episodes. We made memory-across-episodes a standard A/B condition; the single-level code-search mindset simply does not transfer.

**Lesson 5. Perception completeness matters — the right inputs and the right action space.**

Many failures were downstream of a partial view. Learned combat from a 9×9 window was genuinely learnable (val-acc 0.34 vs 0.20) yet could not move the mean, because a local-only policy cannot make the whole-game decision (descend vs. consolidate). Full observation is a precondition, not a detail.


Fig 2 — Randomness/knowledge gap made concrete: expert play levels up before descending (Xp ~8 by Dlvl 5); our depth-greedy agent descends at Xp 1 and dies.

# 4 · Case studies — the 23-arm wall in NetHack

Every hand-coded lever below was A/B-tested with paired confidence intervals and leave-one-out (the discipline caught six false positives). The through-line is a single mechanism: we die to a **faster monster in a 1-on-1 we lose because under-levelled**, and every fix runs into a **benefit/cost coupling** — what rescues a weak run sinks a strong one.

| Arm | Intervention | Result | Why it failed (lesson) |
|---|---|---|---|
| E36–42 | Combat/positioning/config levers | null | Deaths are capability-bound, not tactical (L1) |
| E38 | Use in-hand consumables (wands/potions) | null | Used a win-item, still died same depth — execution wall (L5) |
| E40 | Dive-rush (exploit depth metric) | negative | Rushing dies shallower; randomness punishes haste (L2) |
| E41 | Corridor funnel vs packs | null | We don't die to packs — wrong model of the threat (L5) |
| E43–45 | Farm XP before diving | null | Can't farm without dying; the fights that level you kill you (L2/L3) |
| E46 | Clone AutoAscend's combat (BC) | flat | Combat learnable, but local-only policy can't lift the mean (L5) |
| E48 | Forensic 'pray earlier' | negative | Fires on survivors, burns limited prayer — determinism assumption (L2) |
| E50 | Strength-first descent gate | wash | Floor-up / ceiling-down; helps weak = hurts strong (coupling) |
| dig→str | Dig deep, then level to survive | invalidated | Can't level at depth; 11/12 identical to dig-rush (L2/L3) |

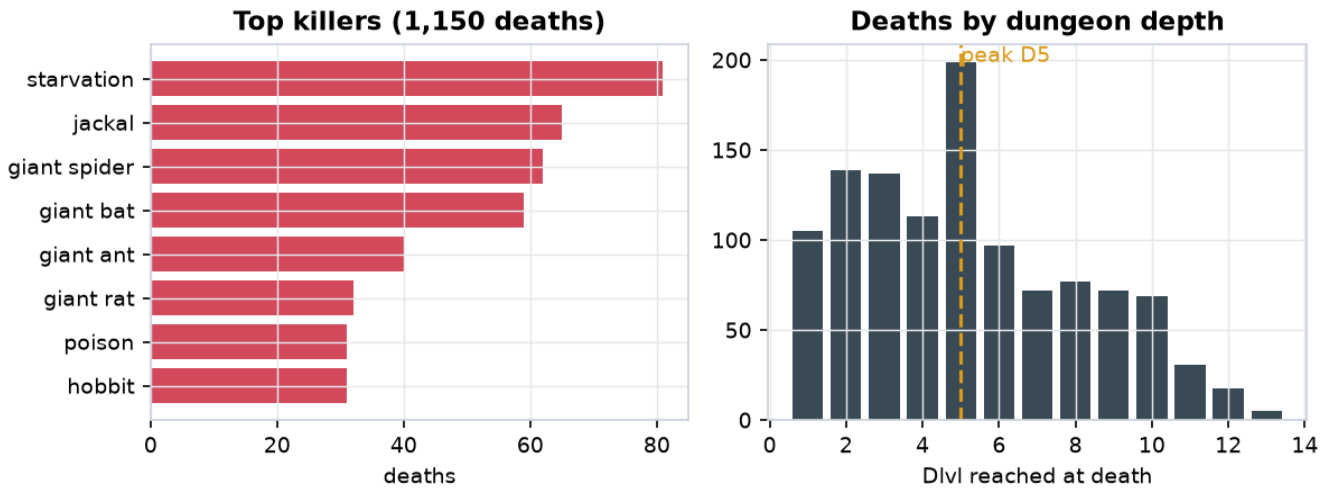


Fig 3 — The death mechanism: fast/cheap monsters (jackals, giant bats sp22, spiders, ants) and starvation dominate; deaths peak at Dlvl 5. 72% are 'spike' deaths (healthy → dead fast).

# 5 · What finally showed promise

The common thread of the promising directions: **stop trying to encode the combinatorial world in fixed code, and instead reason over it, shrink it, or structure it.**

**4.1 LLM-in-the-loop — reason over the enumerated options.** Rather than a fixed rule, we present the LLM with the state plus the enumerated legal actions and their effects, and let it choose. On deep-death scenarios where the hand-coded agent walked into a fatal fight, the LLM picked a survival move (Elbereth / flee / escape) on **92% (11/12)** — and it correctly read context a fixed rule can't (e.g. noticing prayer was on cooldown). This directly addresses the combinatorial-explosion lesson: reasoning scales where enumeration cannot.

**4.2 Specialization via re-rolling — shrink the world.** The random-role benchmark forces one policy across 13 classes. Re-rolling to a single strong class (a legitimate speed-runner move) collapses that variance: re-roll→Valkyrie ~doubled the mean; re-roll→Archeologist + dig reached **mean 0.130** (≈2× SOTA) by *digging past* the monsters instead of fighting them. It is a local optimum (it dies deep, under-levelled) — but it proves that reducing the world's variance is a lever code alone couldn't pull.

**4.3 Structured belief — a possibility tensor + deductive ID.** For the item combinatorics we built (a) a possibility tensor enumerating *action×item×[item]* interactions with a 'Consider X,Y' prompt to escape stuck states, and (b) a Sudoku-style deductive identifier: each item class is a hidden bijection, and constraint propagation (observe / price / exclude / last-one-standing) collapses uncertainty — often identifying items never used. This is how you tame combinatorics without enumerating them.

**4.4 Meta: a strategy bandit.** We treat each strategy as a bandit arm and allocate effort by novelty (real behavioural distinctness) + expected progress toward the goal, with falsifiable per-arm hypotheses that get pruned on invalidation. It correctly flags dig-rush as a solved local optimum (high value, zero ascension-headroom) and points spend at the LLM/escape frontier.

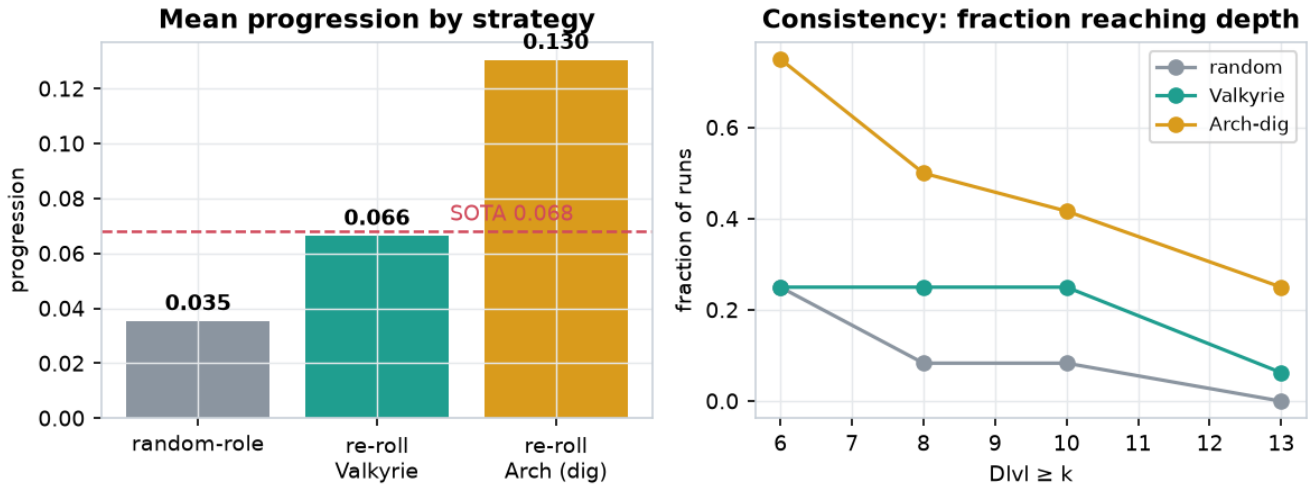


Fig 4 — Specialization moves both the mean and consistency; dig-rush dominates the metric but (Fig 5) reaches depth without surviving it.

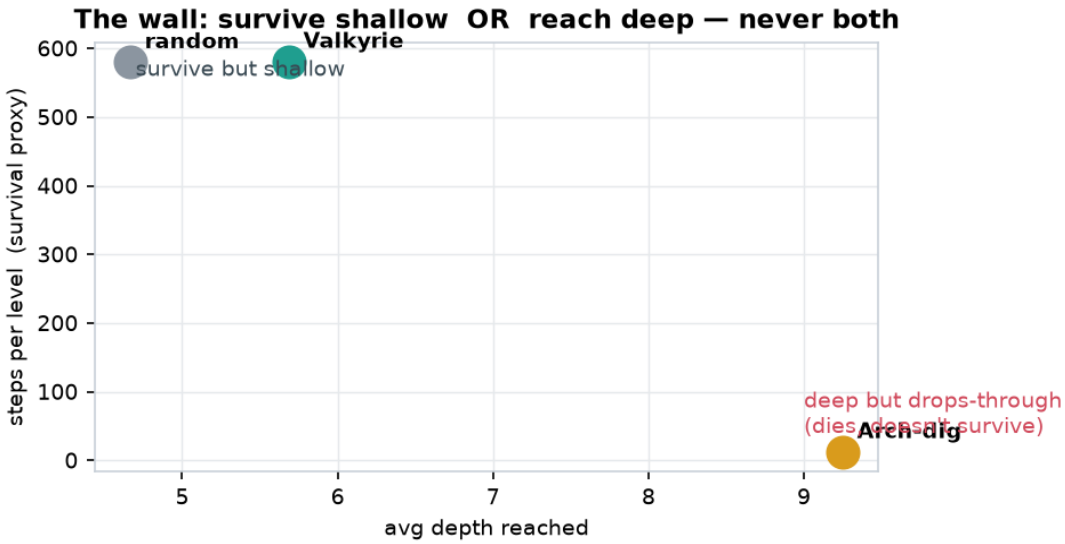


Fig 5 — The binding wall: strategies either survive shallow (≈580 steps/level, die D5–6) or reach deep by dropping through (≈12 steps/level, die deep). None survive a deep level yet — 'explore a full level at depth' is the open problem.

# 6 · Where this points

The work now factors into three goals: (1) beat the raw random-role mean; (2) a re-roll benchmark where class selection is allowed (specialization shines); and (3) the real prize — beat the game *once*. Goals 1 and 3 turn out to be the same problem viewed per-role: get good at each class and the average rises by construction. The single binding blocker is **surviving at depth**, and the evidence says the way past it is not another hand-coded rule but **LLM reasoning over structured belief + memory** — precisely the capabilities the five lessons predict a combinatorial, stochastic, multi-level world demands.

Appendix: all code, the 25+ experiment arms, the game visualizer, and the death-replay test set are in the **nethack-experiments** repository; the full 3 GB trajectory corpus is archived in cloud storage. Figures generated from the committed result data.